



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 07

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 28 de marzo del 2026

CALIFICACIÓN: _____

EJERCICIO REALIZADO

Faro de luz azul

Para esta actividad se pidió trabajar como base el coche que ya hemos jerarquizado y texturizado, en esta ocasión se requería agregar al auto la luz de 1 faro frontal de color azul y posicionar este mismo para que ilumine hacia adelante y se mueva a la par del coche.

Los cambios realizados en código para realizar el ejercicio se presentan a continuación

Window.h

Declaramos los getters que ya hemos utilizado con anterioridad para la rotación de llantas, apertura del coche y traslación de todo nuestro modelo a lo largo del eje Z, además de declarar las variables de estado privadas para cada tipo de transformación

```
public:
    ...
    // =====
    // --- VARIABLES DE ANIMACIÓN DEL COCHE ---
    GLfloat getRotacionLlantas() { return RotLlantas; } // Rotación Llantas
    GLfloat getAperturaCofre() { return ApertCofre; } // Apertura Cofre
    GLfloat getMovCoche() { return MovCoche; } // Traslación Coche en Z
    // =====

private:
    ...
    // =====
    // --- VARIABLES DE ANIMACIÓN DEL COCHE ---
    GLfloat RotLlantas;
    GLfloat ApertCofre;
    GLfloat MovCoche;
    // =====
```

Window.cpp

Inicializamos las variables de movimiento en 0.0f para que al ejecutar nuestro programa, el coche aparezca en nuestra posición original y no salga “volando” fuera de nuestra ventana de visualización.

```
Window::Window() {
    ...
    // =====
    RotLlantas = 0.0f;
    ApertCofre = 0.0f;
    MovCoche = 0.0f;
    // =====
}

Window::Window(GLint windowHeight){
    ...
    // =====
    RotLlantas = 0.0f;
    ApertCofre = 0.0f;
    MovCoche = 0.0f;
    // =====
    ...
}
```

En la función de manejo de teclado asignamos las teclas correspondientes para el movimiento siendo estas F y V sin limitantes para el movimiento del coche y G y B para la apertura del cofre con limitantes

```
void Window::ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode){
    ...
    // =====
    // Llantas y Coche avanza - Teclas H (Adelante) y N (Atrás)
    if (key == GLFW_KEY_F) {
        theWindow->RotLlantas += 5.0f;
        theWindow->MovCoche -= 0.5f;
    }
    if (key == GLFW_KEY_V) {
        theWindow->RotLlantas -= 5.0f;
        theWindow->MovCoche += 0.5f;
    }
    // Cofre - Teclas G (Abrir) y B (Cerrar)
    if (key == GLFW_KEY_G) {
        theWindow->ApertCofre += 2.0f;
        if (theWindow->ApertCofre > 60.0f) theWindow->ApertCofre = 60.0f;
    }
    if (key == GLFW_KEY_B) {
        theWindow->ApertCofre -= 2.0f;
        if (theWindow->ApertCofre < 0.0f) theWindow->ApertCofre = 0.0f;
    }
    // =====
    ...
}
```

Main

Declaramos las instancias de los modelos que conforman nuestro coche, desde la llanta texturizada y el cofre, hasta el modelo restante completo del auto que ya hemos usado, cargando así sus modelos en formato .obj

```
// =====
// Modelos del Coche
Model LlantaT;
Model CofreT;
Model CocheT;
// =====

int main(){
    ...
    // =====
    LlantaT = Model();
    LlantaT.LoadModel("Models/LlantaT1.obj");
    CocheT = Model();
    CocheT.LoadModel("Models/CocheT.obj");
    CofreT = Model();
    CofreT.LoadModel("Models/CofreT.obj");
    // =====
}
```

De las luces con las que ya contábamos, se bajo su intensidad de ambiente y difusa, esto solo con el fin de apreciar mejor nuestro resultado del faro siguiente.

```
// LUZ DE "SOL"
// luz direccional - sólo 1 y siempre debe de existir
mainLight = DirectionalLight(1.0f, 1.0f, 1.0f,
    0.2f, 0.2f, // Bajamos intensidad
    ...

// LUCES PUNTUALES "Tipo antorcha de maincra"
// Declaración de primer luz puntual (LUZ ROJA)
pointLights[0] = PointLight(1.0f, 0.0f, 0.0f,
    0.0f, 0.0f, // Bajamos intensidad
```

```

...
// Luz fija (VERDE)
spotLights[1] = SpotLight(0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, // Bajamos intensidad

```

Esta es la parte más importante de nuestro ejercicio, aquí declaramos una spotLight nueva (índice 2).

- Color: Para dar un poco más de variación se instancio un color azul pálido.
- Intensidad: Como el entorno general se oscureció previamente, se asignaron valores altos de intensidad ambiente (8.0f) y difusa (10.0f) para garantizar que el haz de luz resalte de manera clara sobre los modelos adyacentes.
- Posición inicial: Se dejó sobre el origen, ya que la verdadera posición del faro se recalcularía más adelante frame a frame.
- Dirección inicial: Se fijó en (1.0f, -0.2f, 0.0f), lo que significa que el faro apunta principalmente hacia el frente en el eje X positivo, con una ligera inclinación hacia abajo en el eje Y negativo para simular correctamente cómo un coche ilumina el asfalto.
- Control de distancia (Atenuación): Para evitar que la luz viaje infinitamente y saturé la escena, se configuró la atenuación utilizando la fórmula matemática estándar de iluminación en OpenGL. Se asignó un factor constante de 1.0f, un término lineal de 0.09f y un término cuadrático (o exponente) de 0.032f, estos números se pidieron a una IA, sin embargo, se comprendió que vienen de una tabla “estándar de atenuación” que fue calculada originalmente para el motor gráfico Ogre3D.
- Apertura del cono: Se definió un ángulo de apertura de 80.0f, logrando un cono de luz lo suficientemente ancho para abarcar el frente del coche.

Finalmente, se agregó un contador de spotLight al código para que el shader lo tome en cuenta.

```

// =====
// LUZ SPOT NUEVA: Nuestro faro AZUL
spotLights[2] = SpotLight(0.035f, 0.318f, 0.600f, // Color Azul (RGB)
    8.0f, 10.0f, // Intensidad ambiente y difusa (altas)
    0.0f, 0.0f, 0.0f, // Posición inicial - Sobreescribimos en el while)
    1.0f, -0.2f, 0.0f, // Dirección hacia donde apunta (X, Y, Z)
    1.0f, 0.09f, 0.032f, // Valores de atenuación (constante, lineal, exponente)
    80.0f); // Ángulo de apertura de nuestro cono
spotLightCount++; // Sumamos 1 -> shader sabrá que la luz existe
// =====

```

Dentro de nuestro bucle while por cada ciclo de reloj, calculamos en qué parte del coche debería posicionarse la luz del faro azul, agregando el movimiento en el eje x con cierto dinamismo mediante nuestro getter para mover el coche. Es importante mencionar que, ya que la posición en Y y en Z no cambiaría, pudimos dejarlas establecidas antes de nuestro while en cada ciclo de reloj, pero se decidió dejar dentro de este de forma momentánea para tener bien ubicados donde movemos nuestra spotlight respecto al coche. Además, se

utilizó la función SetPos, que nos permite sobrescribir la memoria de la lyz en su variable interla protegida llamada position.

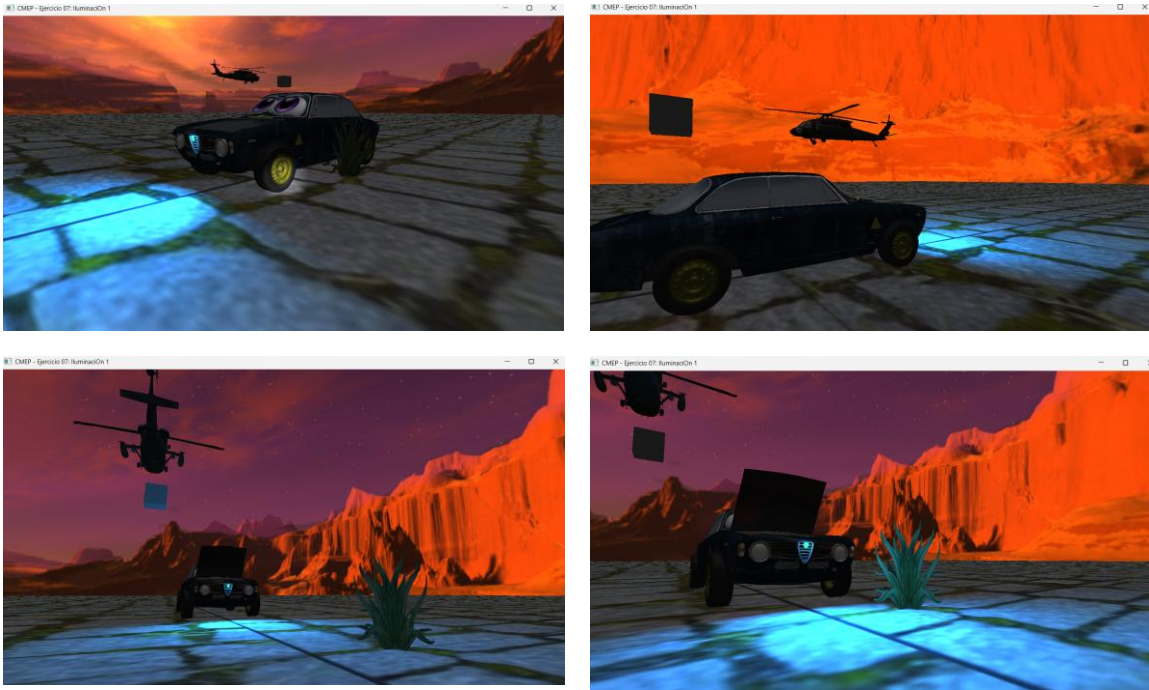
```
while (!mainWindow.getShouldClose()){  
    ...  
    // =====  
    // ACTUALIZAMOS LA POSICIÓN DE NUESTRO FARO AZUL  
    // --- Aquí lo vinculamos al coche :3  
    GLfloat faroPosEn_X = 8.8f + mainWindow.getMovCoche();  
    GLfloat faroPosEn_Y = 1.0f; // Altura del faro  
    GLfloat faroPosEn_Z = 0.0f; // Profundidad del faro  
    // Aplicamos la nueva posición a la luz 2  
    spotLights[2].SetPos(glm::vec3(faroPosEn_X, faroPosEn_Y, faroPosEn_Z));  
    // =====
```

Finalmente instanciamos y jerarquizamos nuestro coche como en prácticas anteriores.

```
// =====  
//Instancia del coche  
model = glm::mat4(1.0);  
// Agregamos el movimiento del coche en Z al padre  
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getMovCoche(), -1.1f, 0.0f));  
modelaux = model; // Pivote central guardado  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
CocheT.RenderModel();  
// Llanta trasera derecha -----  
model = modelaux;  
model = glm::translate(model, glm::vec3(-4.3f, 0.4f, 3.1f));  
// Rotación de la llanta (RotLlantas) ANTES de acomodarla para que gire en su eje  
model = glm::rotate(model, glm::radians(mainWindow.getRotacionLlantas()), glm::vec3(0.0f, 0.0f, 1.0f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
color = glm::vec3(0.5f, 0.5f, 0.5f); //Llanta con color gris  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
LlantaT.RenderModel();  
// Llanta trasera izquierda -----  
model = modelaux;  
model = glm::translate(model, glm::vec3(-4.3f, 0.4f, -3.1f));  
model = glm::rotate(model, glm::radians(mainWindow.getRotacionLlantas()), glm::vec3(0.0f, 0.0f, 1.0f));  
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
LlantaT.RenderModel();  
// Llanta delantera derecha -----  
model = modelaux;  
model = glm::translate(model, glm::vec3(6.0f, 0.4f, 3.1f));  
model = glm::rotate(model, glm::radians(mainWindow.getRotacionLlantas()), glm::vec3(0.0f, 0.0f, 1.0f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
LlantaT.RenderModel();  
// Llanta delantera izquierda -----  
model = modelaux;  
model = glm::translate(model, glm::vec3(6.0f, 0.4f, -3.1f));  
model = glm::rotate(model, glm::radians(mainWindow.getRotacionLlantas()), glm::vec3(0.0f, 0.0f, 1.0f));  
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
LlantaT.RenderModel();  
// Cofre del coche -----  
model = modelaux;  
model = glm::translate(model, glm::vec3(4.2f, 2.6f, 0.0f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
// Rotación del cofre para abrir/cerrar (articulacion2)  
model = glm::rotate(model, glm::radians(mainWindow.getAperturaCofre()), glm::vec3(1.0f, 0.0f, 0.0f));  
color = glm::vec3(0.5f, 0.5f, 0.5f); //Llanta con color gris  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
CofreT.RenderModel();  
// =====
```

EJECUCIÓN

Teclas de movimiento: F,V (coche) y G,B (cofre)



Para una mejor visualización se adjunta un link al GIF de drive.

https://drive.google.com/file/d/1vEzpmjgZEE91K_xWaLFBDWQK1I0uzWNz/view?usp=sharing

PROBLEMAS PRESENTADOS

Para este ejercicio no se presentó algún problema, fue relativamente sencillo al exportar código ya hecho, modelos y texturizado.

CONCLUSIONES

Sobre el ejercicio de clase

La complejidad de este ejercicio radicó en pasar de una iluminación ambiental estática a implementar fuentes de luz dinámicas, como una luz focal (SpotLight), requirió un análisis detallado de los parámetros de atenuación (constante, lineal y cuadrática) para lograr un decaimiento (que después supimos que también es llamado falloff) realista en el espacio tridimensional.

Comentarios generales

Como sugerencia nuestra segunda parte de la práctica de iluminación resultaría de gran utilidad proporcionarnos una tabla de referencia rápida o material visual que relacione los valores matemáticos de atenuación con distancias aproximadas en las unidades del mundo de OpenGL.

REFERENCIAS

- De Vries, J. (s.f.). *Light casters*. LearnOpenGL. Recuperado el 28 de marzo de 2026, de <https://learnopengl.com/Lighting/Light-casters>
- G-Truc Creation. (s.f.). *OpenGL Mathematics (GLM)*. Recuperado el 28 de marzo de 2026, de <https://glm.g-truc.net/>